



UNIVERSITAS
BRAWIJAYA



brone
BRAWIJAYA ONLINE LEARNING



Fondasi Aljabar Linear untuk *Computational Modelling*

Course:

Matrikulasi S3 Ilmu Komputer FILKOM UB

by:

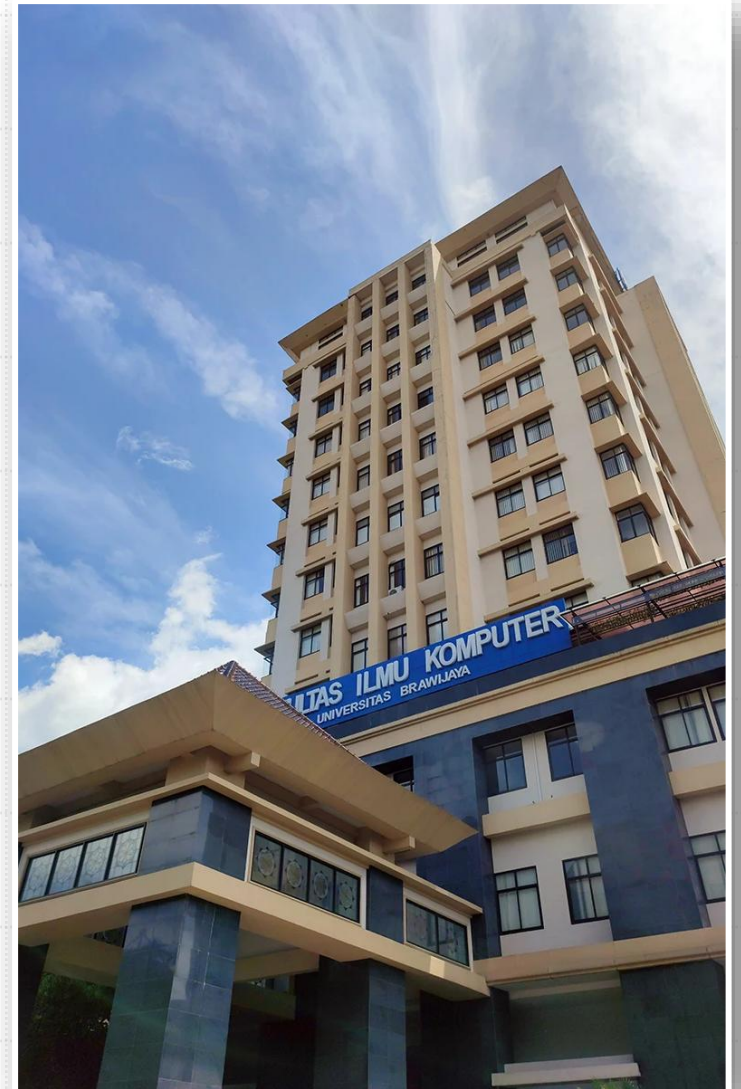
Yuita Arum Sari, S.Kom., M.Kom., Ph.D.

yuita@ub.ac.id

Versi Dokumen:

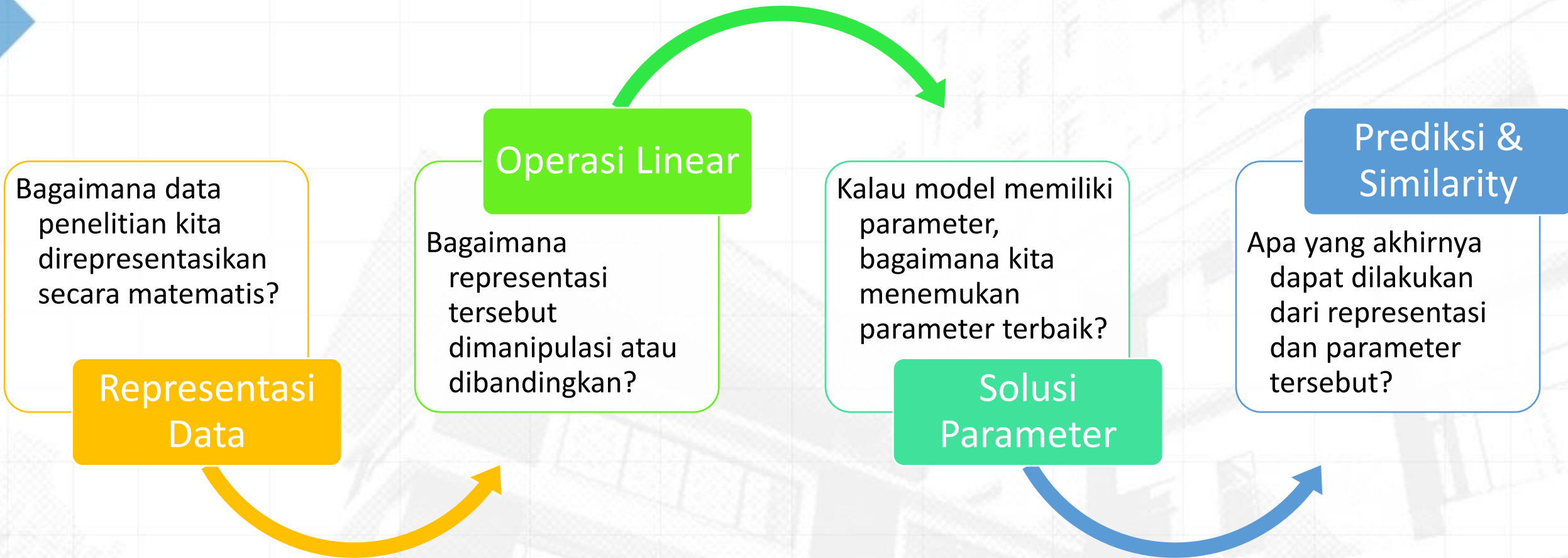
Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id

From Data to Model: The Mathematics Behind Computational Intelligence



Mengapa Aljabar Linear Penting untuk AI/ML?

Banyak algoritma AI/ML dapat dilihat sebagai **operasi pada vector, matrix, dan tensor**.



memprediksi **skor kematangan buah** berdasarkan dua fitur citra.

Setiap buah direpresentasikan dengan 2 fitur hasil pengolahan citra:

- x_1 = **rata-rata warna** (color)
- x_2 = **tekstur** (texture)

Misalnya 3 buah:

$$X = \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.5 \\ 0.3 & 0.8 \end{bmatrix}$$

$X \in \mathbb{R}^{3 \times 2}$

3 baris = 3 buah

2 kolom = 2 fitur citra

Model memberikan bobot pada masing-masing fitur:

$$\theta = \begin{bmatrix} 70 \\ 30 \end{bmatrix}$$

$\theta \in \mathbb{R}^2$

Artinya:

- bobot fitur warna = 70
- bobot fitur tekstur = 30

Prediction — $\hat{y} = X\theta$

$$\hat{y} = \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.5 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 70 \\ 30 \end{bmatrix} = \begin{bmatrix} 62 \\ 57 \\ 45 \end{bmatrix}$$

Sehingga:

Buah	Color	Texture	Prediksi skor
A	0.8	0.2	62
B	0.6	0.5	57
C	0.3	0.8	45

Mengapa Aljabar Linear Penting untuk AI/ML? (2)

- Data digital (gambar, teks, video) tidak langsung menjadi prediksi.
- Data tersebut terlebih dahulu **direpresentasikan sebagai fitur numerik**, kemudian model mengombinasikan fitur tersebut menggunakan parameter θ .

Citra

matrix/tensor intensitas piksel atau *feature map*

Recommender System

user dan item sebagai vector latent

NLP

dokumen sebagai vector TF-IDF atau embedding

Machine Learning

training = mencari parameter yang meminimalkan error

“model learning” sering dimulai dari pertanyaan sederhana → parameter mana yang paling cocok dengan data?

Mengapa Ini Relevan Dipelajari?

Peneliti perlu mampu membaca model sebagai struktur matematis, bukan sekadar memanggil fungsi library.

1

Memahami Asumsi

Agar tidak hanya menjalankan library, tetapi memahami asumsi dan konsekuensi setiap model.

2

Merancang Eksperimen

Agar dapat merancang eksperimen yang kuat: baseline, ablation, metric, dan error analysis.

3

Menjelaskan Novelty

Agar dapat menjelaskan *novelty* metode secara lebih ilmiah dan presisi.

4

Menghubungkan Fondasi

Agar dapat menghubungkan AI modern dengan fondasi numerik: matrix, optimization, representation learning.

Good research = correct formulation + meaningful representation + rigorous evaluation

Langkah-langkah dari Matrix ke Model Prediktif

1. Representasi

1. Scalar, vector, matrix, tensor

2. Geometri Vector

2. Vector operations
3. Norm & distance
4. Dot product
5. Cosine similarity

3. Matrix dan Sistem Linear

6. Matrix multiplication sbg transformasi
7. Sistem $Ax = b$
8. Square vs overdetermined

4. Rank & Invertibility

9. Rank & linear independence
10. Mengapa A^{-1} tidak selalu ada

5. Least Squares & Regresi

11. Least squares
12. Normal equation
13. Pseudo-inverse
14. Linear regression
15. Residual & error

6. Connection to AI/ML

16. $X\beta \approx y$ & similarity sebagai bahasa umum model

1. Representasi Data

Scalar, vector, matrix, tensor: bahasa numerik dasar AI/ML

Scalar, Vector, Matrix, Tensor

Empat bentuk representasi numerik yang paling sering muncul dalam AI/ML.

Scalar	Vector	Matrix	Tensor
0-D	1-D	2-D	n-D
1 nilai tunggal	daftar nilai (arah + panjang)	tabel baris × kolom	array multi-dimensi

Contoh kalimat: “Aku suka AI”

Bentuk	Dimensi	Contoh Representasi	Makna
Scalar	0-D	0.9	Satu nilai, misalnya skor sentimen
Vector	1-D	[0.25, −, 0.81, 0.400.05]	Satu kata → embedding
Matrix	2-D	3 × 4	3 kata × 4 nilai embedding
Tensor	3-D+	32 × 10 × 300	32 kalimat × 10 kata × 300 embedding

Dalam machine learning, hampir semua objek akhirnya direpresentasikan sebagai angka.

Perbedaan bentuk data menentukan operasi matematis dan komputasi yang digunakan.

Dimensi Data: Dari Angka Tunggal ke Tensor Batch

Semakin Tinggi Dimensi → Semakin Kompleks

Scalar

5

Vector (d,)

[5, 3, 8]

Matrix (n,d)

n baris × d kolom

Tensor (B,H,W,C)

batch citra 4-D

Contoh dalam AI/ML

1

Scalar

satu nilai loss, satu prediksi probabilitas kelas

2

Vector

satu baris fitur: dokumen, citra ringkas, atau user profile

3

Matrix

seluruh dataset tabular X, atau satu citra grayscale (H×W)

4

Tensor

batch citra berwarna (Batch × Height × Width × Channel), video, aktivasi *hidden layer*

Contoh Representasi Vector dalam AI

Satu objek data apa pun dapat direpresentasikan sebagai vector fitur $x \in \mathbb{R}^d$.

Jenis Data	Fitur yang Direpresentasikan	Contoh Vektor x	Makna
Teks	Bobot kata (TF-IDF)	$\begin{bmatrix} 0.85 \\ 0.42 \\ 0.00 \end{bmatrix}$	AI = 0.85, ML = 0.42, CV = 0
Citra	Warna, tekstur, edge, dll.	$\begin{bmatrix} 180 \\ 50 \\ 0.75 \\ 12.4 \end{bmatrix}$	Rata-rata R=180, G=50, tekstur=0.75, edge=12.4
Tabular	Atribut objek/subjek	$\begin{bmatrix} 21 \\ 3.80 \\ 8 \end{bmatrix}$	Usia=21, IPK=3.80, aktivitas=8

Apa arti x ? $x \in \mathbb{R}^d$

- x = representasi numerik **satu objek**
- d = jumlah fitur/dimensi
- Setiap angka = **satu aspek yang digunakan untuk menggambarkan objek**
- Setelah menjadi vektor, data dapat diproses dengan operasi yang sama: **dot product, norm, similarity, transformasi, dan prediksi.**

Dari Feature Engineering → Feature Learning

Satu objek data apa pun dapat direpresentasikan sebagai vector fitur $x \in \mathbb{R}^d$.

Machine Learning tradisional

Data → Feature Engineering → x → Model

Fitur dirancang/diekstraksi oleh manusia.

Deep Learning

Data → Neural Network → x_{learned} → Prediction

Model belajar representasi fitur secara otomatis melalui berbagai layer.

Model tidak bekerja langsung pada “teks”, “gambar”, atau “mahasiswa”.
Model bekerja pada representasi numeriknya.

2. Geometri Vector

Operasi, *norm*, *distance*, *dot product*, dan *cosine similarity*

Vector Operations: Addition dan Scaling

Operasi sederhana tetapi menjadi dasar embedding arithmetic dan transformation.

Penjumlahan Vector

$$u + v = [u_1 + v_1, u_2 + v_2]^T$$

Digabung komponen-per-komponen; secara geometris, disusun ujung-ke-ujung (head-to-tail).

Perkalian Skalar (Scaling)

$$c \cdot u = [c \cdot u_1, c \cdot u_2]^T$$

Memperpanjang, memperpendek, atau membalik arah vector (jika c negatif); arah tetap kolinear.

Interpretasi Geometris

- Vector bukan hanya daftar angka; vector memiliki arah, panjang, dan posisi relatif dalam ruang fitur.
- Penjumlahan vector merepresentasikan kombinasi dua sinyal/fitur (mis. menggabungkan dua embedding).
- *Scaling* merepresentasikan penguatan atau pelemahan sinyal tanpa mengubah arah informasinya.

Operasi Vektor: Menggabungkan dan Memberi Bobot

Misalkan kita ingin merepresentasikan kalimat:

“AI sangat bagus”

Anggap embedding sederhananya:

$$u_{\text{AI}} = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, v_{\text{bagus}} = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

1. Penjumlahan → Menggabungkan informasi

$$u_{\text{AI}} + v_{\text{bagus}} = \begin{bmatrix} 0.8 + 0.3 \\ 0.2 + 0.7 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 0.9 \end{bmatrix}$$

Makna: dua representasi digabungkan menjadi representasi yang membawa informasi dari keduanya.

2. Perkalian Skalar → Memberikan bobot

Misalkan kata **“bagus”** dianggap lebih penting, sehingga diberi bobot $c = 2$:

$$2v_{\text{bagus}} = 2 \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 1.4 \end{bmatrix}$$

Kemudian gabungkan dengan **AI**:

$$u_{\text{AI}} + 2v_{\text{bagus}} = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.6 \\ 1.4 \end{bmatrix} = \begin{bmatrix} 1.4 \\ 1.6 \end{bmatrix}$$

Model komputasi tidak hanya menyimpan angka; model memanipulasi representasi numerik melalui operasi matematis.

Vector Operations: Perhitungan Manual

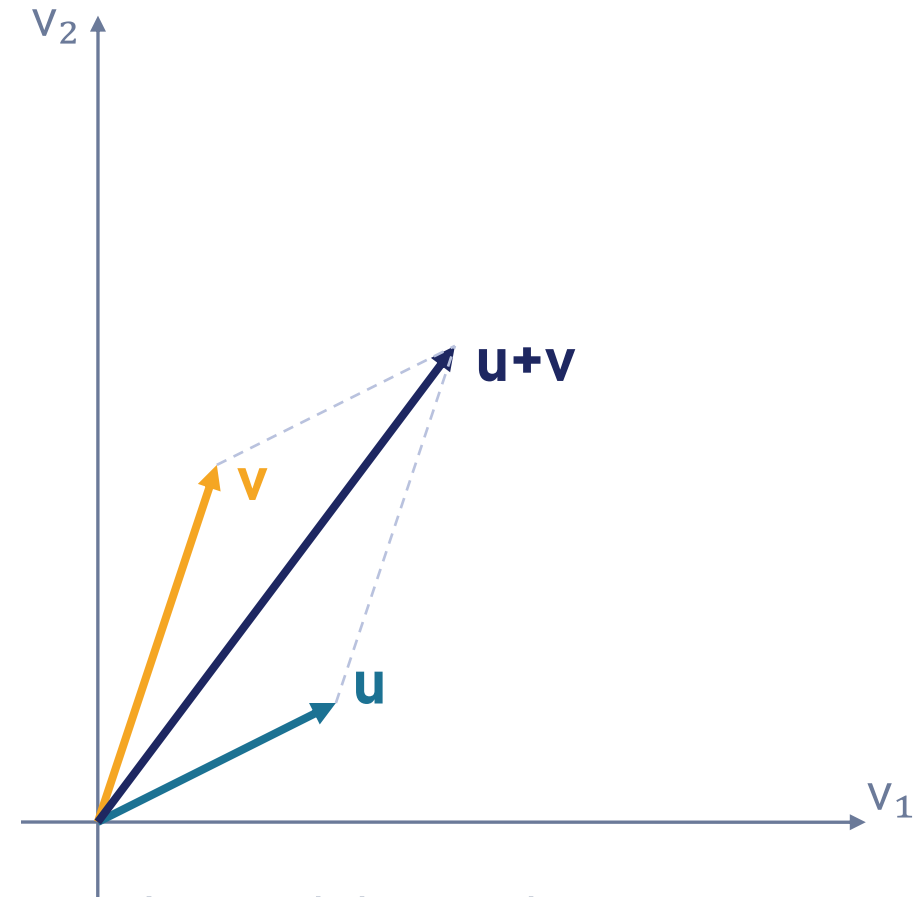
$$u = [2,1]^T, \quad v = [1,3]^T$$

$$u + v = [2 + 1, 1 + 3]^T = [3,4]^T$$

$$2u = [2 \times 2, 2 \times 1]^T = [4,2]^T$$

Bacaan Diagram →

- u dan v digambar sebagai panah dari titik asal $(0,0)$.
- $u+v$ adalah diagonal jajargenjang yang dibentuk oleh u dan v (aturan jajargenjang).
- $2u$ searah dengan u tetapi dua kali lebih panjang.



Aturan jajargenjang: $u + v$

Norm: Mengukur Panjang Vector

Norm sering digunakan untuk distance, normalization, regularization, dan gradient.

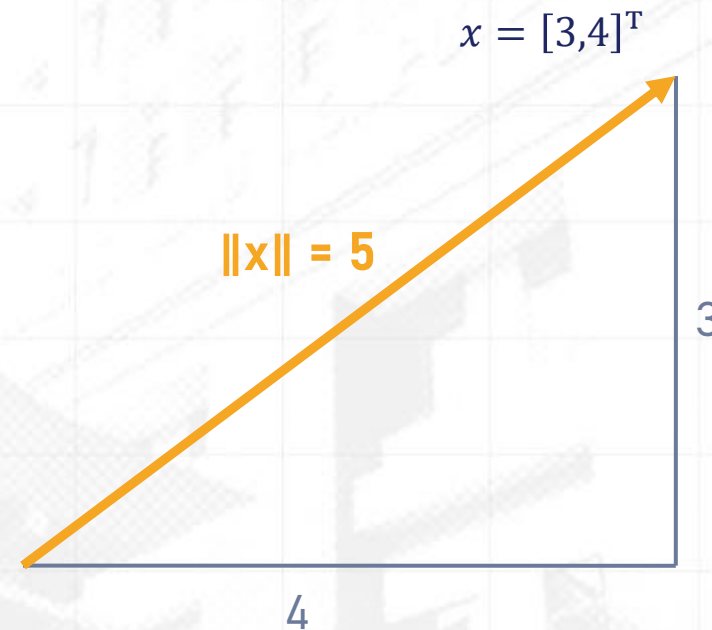
$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$$

$$x = [3, 4]^T \Rightarrow \|x\|_2 = \sqrt{9 + 16} = \sqrt{25} = 5$$

Kegunaan dalam AI/ML

- Norm membantu membandingkan skala vector.
- Regularisasi L2 (ridge) memakai norm parameter $\|\theta\|^2$.
- Dalam embedding, normalization membuat perbandingan lebih stabil.

Segitiga 3-4-5 (Pythagoras)



Membandingkan Skala Vektor

Skenario: Mesin pencari menghitung frekuensi kata kunci "AI" dan "Robot" di dua dokumen berbeda.

- **Dokumen A** (pendek): $A = [3, 4]^T$
- $\|A\|_2 = \sqrt{3^2 + 4^2} = 5$
- **Dokumen B** (panjang): $B = [30, 40]^T$
- $\|B\|_2 = \sqrt{30^2 + 40^2} = 50$

Penerapan: Walaupun topik utamanya identik karena memiliki rasio kata 3:4, nilai norm menunjukkan bahwa **Dokumen B memiliki skala frekuensi kata 10 kali lebih besar dibandingkan Dokumen A**. Hal ini terjadi karena jumlah kemunculan kata dalam dokumen lebih panjang.

Regularisasi L2

Skenario: Model AI memprediksi apakah sebuah email termasuk spam berdasarkan bobot parameter $\theta = [\theta_1, \theta_2]^T$ untuk kata "Diskon" dan "Halo".

Tanpa L2:

Model dapat memberikan bobot ekstrem, misalnya $\theta = [1000, 0.1]^T$. Jika terdapat email biasa yang secara tidak sengaja memuat kata "Diskon", email tersebut berpotensi langsung diklasifikasikan sebagai spam.

Dengan L2:

Fungsi *loss* menambahkan penalti berupa kuadrat norm parameter:

$$\|\theta\|_2^2 = 1000^2 + 0.1^2 = 1.000.000,01$$

Nilai penalti tersebut sangat besar.

Penalti L2 mendorong model untuk menghindari bobot yang terlalu besar. Sebagai ilustrasi, bobot dapat berubah menjadi $\theta = [2, 1]^T$, sehingga:

$$\|\theta\|_2^2 = 2^2 + 1^2 = 5$$

model cenderung tidak terlalu bergantung pada satu kata kunci dan dapat menghasilkan prediksi yang lebih stabil. **Nilai bobot akhir merupakan hasil proses optimasi**, bukan nilai yang selalu dipaksakan menjadi $[2, 1]^T$.

Normalisasi Embedding

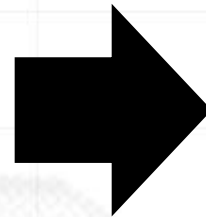
Skenario: Sistem pengenalan wajah menggunakan dua fitur numerik, yaitu jarak antara kedua mata dan lebar wajah. Sistem membandingkan hasil pengukuran dua orang untuk melihat kemiripan proporsi wajah.

Orang A: $A = [8, 6]^T$

$$\|A\|_2 = \sqrt{8^2 + 6^2} = \sqrt{100} = 10$$

Orang B: $B = [7, 5]^T$

$$\|B\|_2 = \sqrt{7^2 + 5^2} = \sqrt{74} \approx 8,60$$



Normalisasi Orang A:

$$A/\|A\|_2 = [8/10, 6/10]^T = [0,8; 0,6]^T$$

Normalisasi Orang B:

$$B/\|B\|_2 = [7/8,60, 5/8,60]^T \approx [0,814; 0,581]^T$$

Menghitung Jarak City Block

Jarak City Block dihitung dengan menjumlahkan selisih absolut setiap elemen:

$$d(A, B) = |A_1 - B_1| + |A_2 - B_2|$$

Dengan menggunakan hasil normalisasi:

$$d(A, B) = |0,800 - 0,814| + |0,600 - 0,581| = 0,014 + 0,019 \approx 0,033$$

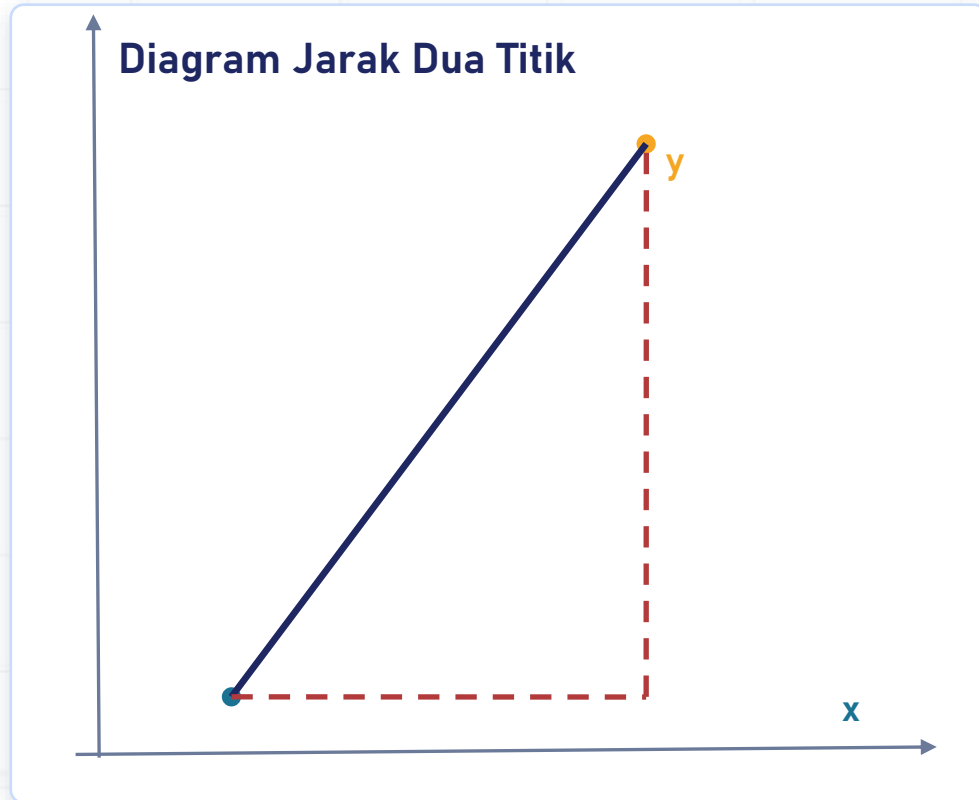
Jarak kecil $\rightarrow$ embedding semakin dekat $\rightarrow$ karakteristik wajah yang direpresentasikan semakin mirip.

Normalisasi membuat representasi wajah berada pada skala yang sama sehingga jarak antarembding lebih adil untuk dibandingkan.



Distance: Seberapa Jauh Dua Data?

Distance mengukur kedekatan titik dalam ruang fitur.



- **Euclidean Distance:** Garis lurus terpendek dari x ke y .
- **Manhattan Distance:** Jarak dengan menjumlahkan selisih horizontal dan vertikal, seperti menyusuri blok jalan.

Euclidean

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Contoh: $x = (1, 2), y = (4, 6) \rightarrow d = 5$

Manhattan

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

Contoh: $x = (1, 2), y = (4, 6) \rightarrow d=7$

Distance mengukur seberapa berbeda dua representasi data.
Pemilihan distance bergantung pada karakteristik datanya.

Minkowski

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

Contoh: $p = 3$, $x = (1, 2)$, $y = (4, 6) \rightarrow d = \sqrt[3]{91} \approx 4.50$

Chebyshev

$$d(x, y) = \max_i |x_i - y_i|$$

Contoh: $x = (1, 2)$, $y = (4, 6) \rightarrow d = 4$

Cosine

$$d(x, y) = 1 - \frac{x \cdot y}{\|x\|_2 \|y\|_2}$$

Contoh: $x = (1, 0)$, $y = (0, 1) \rightarrow d = 1$

Jaccard

$$d(X, Y) = 1 - \frac{|X \cap Y|}{|X \cup Y|}$$

Contoh: $X = \{A, B, C\}$, $Y = \{B, C, D\} \rightarrow d = 1 - \frac{2}{4} = 0.5$

Hamming

$$d(x, y) = \sum_{i=1}^n \mathbf{1}(x_i \neq y_i)$$

Contoh: $x = 101101$, $y = 100001 \rightarrow d=2$

Representasi data berbeda $\Rightarrow$
cara mengukur kemiripan/jarak juga berbeda

Normalization: Mengubah Panjang, Mempertahankan Arah

Sering dilakukan sebelum cosine similarity atau gradient-based learning.

$$x_{norm} = \frac{x}{\|x\|_2}$$

Hitung panjang vektor $x = [3,4]^T$:

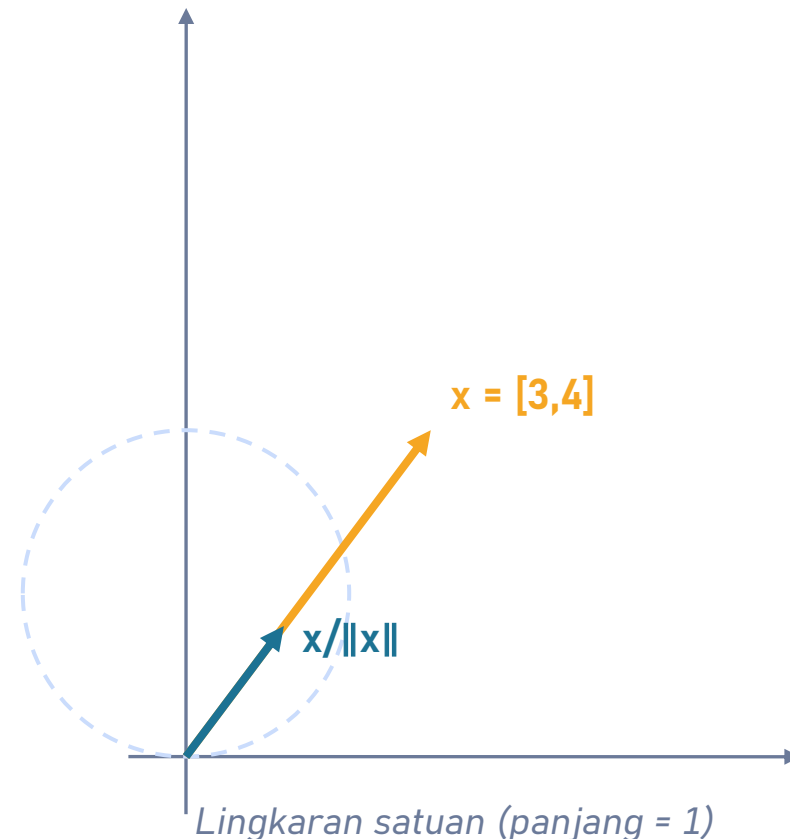
$$\|x\|_2 = \sqrt{3^2 + 4^2} = 5$$

Bagi setiap elemen vektor dengan nilai panjangnya (5):

$$x_{norm} = \begin{bmatrix} 3/5 \\ 4/5 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}^T$$

Jika panjang vektor x_{norm} dihitung ulang, hasilnya tepat bernilai 1 ($\sqrt{0.6^2 + 0.8^2} = 1$).

Sebelum vs Sesudah Normalisasi



Normalization: Mengapa ini Penting dalam AI/ML

Kasus: klasifikasi kualitas buah dari citra

Misalkan model menggunakan dua fitur:

$$x = \begin{bmatrix} \text{Mean RGB} \\ \text{Texture} \end{bmatrix} = \begin{bmatrix} 200 \\ 0.8 \end{bmatrix}$$

Masalahnya: **skala berbeda jauh**.

Jika langsung digunakan, fitur RGB dapat **mendominasi perhitungan** bukan karena lebih informatif, tetapi karena angkanya lebih besar.

Solusi → Normalisasi. Ubah fitur ke skala yang sebanding, misalnya:

$$x_{\text{norm}} = \begin{bmatrix} 0.78 \\ 0.80 \end{bmatrix}$$

Sekarang model dapat mempertimbangkan **warna dan tekstur secara lebih seimbang**.

Menyeimbangkan fitur → fitur dengan skala besar tidak mendominasi.

Menstabilkan training → optimisasi model lebih stabil.

Membandingkan data → fokus pada arah/pola, bukan besar nilai vektornya.

Dot Product: Perkalian yang Mengandung Geometri

Dot product mengukur keselarasan arah dua vector.

Bentuk Aljabar

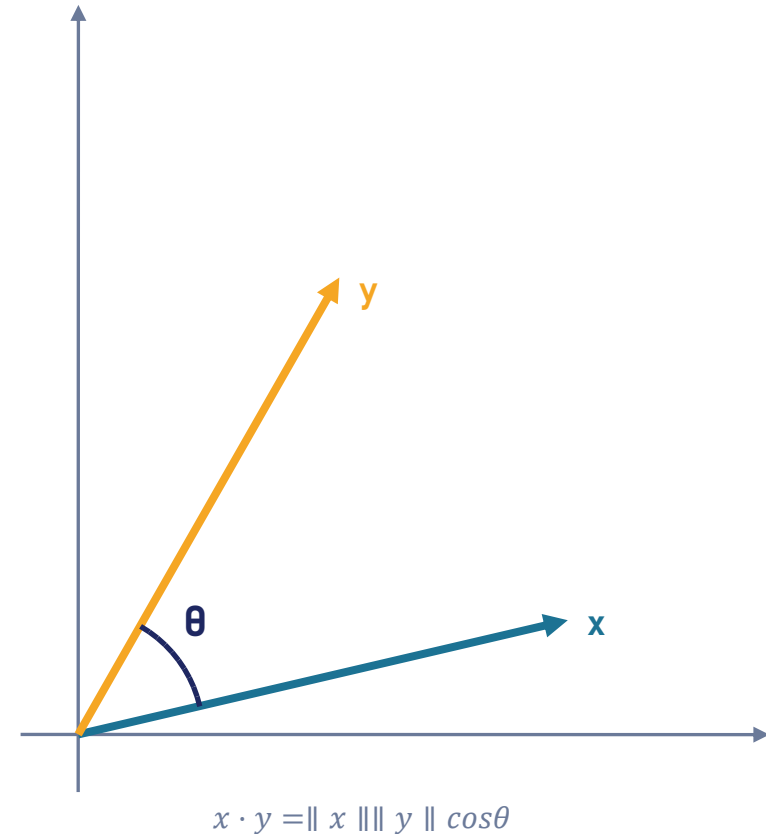
$$x \cdot y = x^T y = \sum_i x_i y_i$$

Bentuk Geometris

$$x \cdot y = \|x\| \|y\| \cos \theta$$

Sudut	Nilai	Makna
$\theta < 90^\circ$	> 0	Arah relatif sejalan
$\theta = 90^\circ$	$= 0$	Tegak lurus
$\theta > 90^\circ$	< 0	Arah relatif berlawanan

Sudut Antara Dua Vector



Penerapan Dot Product dalam Transformer

$$\text{Attention Score} = \frac{QK^T}{\sqrt{d_k}}$$

- **Q (Query)**: informasi yang sedang dicari
- **K (Key)**: informasi yang dibandingkan
- QK^T : **dot product** → mengukur kecocokan Query dan Key
- Skor → **Softmax** → menentukan bobot perhatian
- Bobot digunakan untuk menggabungkan **V (Value)**

Dot product membantu Transformer menentukan token mana yang paling relevan terhadap token yang sedang diproses.

Contoh: dalam “*Saya makan karena lapar*”, representasi “**lapar**” dapat memberikan perhatian lebih besar pada “**makan**” karena keduanya relevan secara konteks.

Dot Product: Contoh Perhitungan Manual

Gunakan vector dokumen sederhana dari tiga kata kunci: AI, ML, Vision.

$$d_1 = [3, 2, 0], \quad d_2 = [2, 3, 1]$$

$$d_1 \cdot d_2 = (3)(2) + (2)(3) + (0)(1) = 12$$

1. Kesamaan konten

Vektor frekuensi kata

$$d_1 = [3, 2, 0]$$

$$d_2 = [2, 3, 1]$$

$$d_1 \cdot d_2 = 13$$

Kedua dokumen memiliki kata AI dan ML yang sama-sama sering muncul.

Solusi: Cosine Similarity

2. Pengaruh panjang

Frekuensi kata memengaruhi skor

Pendek $[1, 1]$

Panjang $[10, 10]$

$$[1, 1] \cdot [1, 1] = 2$$

$$[10, 10] \cdot [10, 10] = 200$$

Skor lebih besar belum tentu berarti topiknya lebih mirip.

$$\cos \theta = \frac{x \cdot y}{\|x\| \|y\|}$$

Mengukur kemiripan arah vektor dengan menormalisasi panjangnya, sehingga pengaruh ukuran dokumen dapat dikurangi.

Cosine Similarity: Fokus pada Arah Vector

Sangat populer untuk *document retrieval*, *image retrieval*, dan *embedding search*.

$$\text{cosine}(x, y) = \frac{x^T y}{\|x\|_2 \|y\|_2}$$

$$\cos \approx 1$$

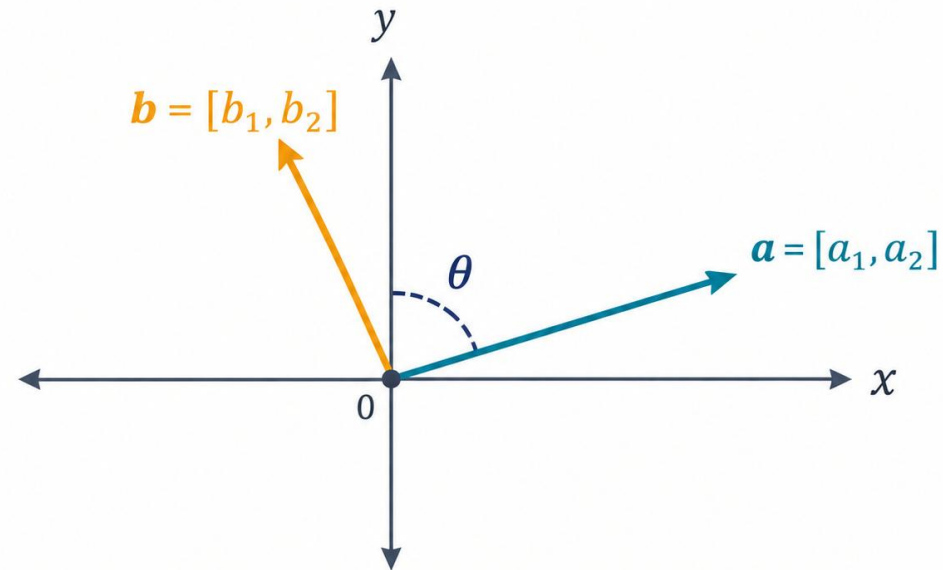
arah sangat mirip

$$\cos \approx 0$$

arah hampir tegak lurus

$$\cos < 0$$

arah berlawanan (sudut tumpul)



$$a \cdot b = \|a\| \|b\| \cos \theta$$

Sudut tumpul ($\theta > 90^\circ$) $\rightarrow$ $\cos \theta$ negatif $\rightarrow$
arah a dan b cenderung berlawanan.

Cosine Similarity: Contoh Manual

$$a = [4,1]^T, \quad b = [-1,3]^T$$

1) Dot product

$$a \cdot b = 4(-1) + 1(3) = -4 + 3 = -1$$

2) Norma masing-masing

$$\|a\| = \sqrt{16 + 1} = \sqrt{17} \approx 4,123$$

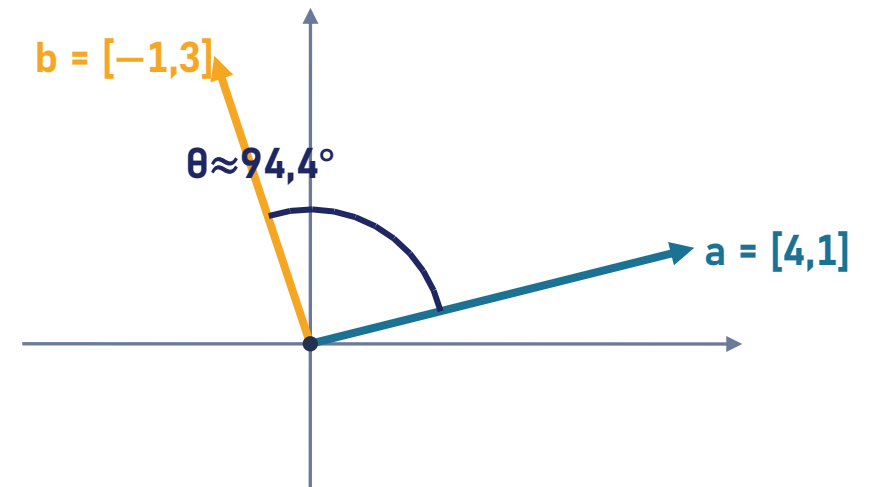
$$\|b\| = \sqrt{1 + 9} = \sqrt{10} \approx 3,162$$

3) Cosine similarity

$$\cos\theta = -1/(4,123 \times 3,162) \approx -0,077$$

$$\theta = \arccos(-0,077) \approx 94,4^\circ$$

Visualisasi Sudut a dan b



Sudut tumpul ($>90^\circ$) $\rightarrow \cos\theta$ negatif $\rightarrow$ arah a dan b cenderung berlawanan.

Solve-the-Case: Document and Image Similarity

Contoh kecil sebelum menggunakan embedding modern: prinsipnya identik.

A. Document Retrieval

$$q = [1,1,0], d_1 = [3,2,0], d_2 = [0,1,4]$$

$$\cos(q, d_1) = 5 / (\sqrt{2}\sqrt{13}) \approx \mathbf{0,981}$$

$$\cos(q, d_2) = 1 / (\sqrt{2}\sqrt{17}) \approx \mathbf{0,171}$$

d_1 jauh lebih relevan dibanding d_2 untuk query q

Query q berisi topik AI dan ML — d_1 juga membahas AI dan ML, sehingga arahnya sangat sejalan.

B. Image Retrieval

Vector fitur citra dapat dibandingkan seperti vector dokumen.

Citra A

Citra B

Citra C

$[mean_R, mean_G, mean_B, texture]$ per citra

$$sim(A,B) > sim(A,C) \Rightarrow B \text{ lebih mirip } A$$

- Computer vision klasik: fitur berupa warna, tekstur, bentuk, atau edge.
- Deep learning: fitur berupa embedding dari CNN/ ViT/CLIP, matematikanya tetap sama.

3. Matriks dan Sistem Linear

Matrix sebagai dataset, transformasi, dan formulasi $Ax = b$

Matrix sebagai Dataset

Dataset tabular dapat ditulis sebagai matrix X .

$$X = [x_1^T; x_2^T; \dots; x_n^T] \in \mathbb{R}(n \times d)$$

- n = jumlah sampel/observasi
- d = jumlah fitur/variabel

- Baris matrix: satu objek data.
- Kolom matrix: satu fitur.
- Target prediksi ditulis sebagai vector $y \in \mathbb{R}^n$.
- Training model = mencari parameter θ yang menghubungkan X dengan y .

Struktur Matrix X

	fitur ₁	fitur ₂	fitur ₃
x_1^T	0.2	1.4	3.1
x_2^T	0.5	2.1	2.8
x_3^T	0.1	0.9	3.6
$\vdots$	$\vdots$	$\vdots$	$\vdots$

n baris (sampel) $\times$ d kolom (fitur)

$$y \in \mathbb{R}^n(\text{target/label})$$

Setiap baris X berpasangan dengan satu nilai y (regresi) atau satu label (klasifikasi).

Matrix Multiplication: Menggabungkan Fitur dan Parameter

Perkalian matrix-vector menghasilkan prediksi untuk semua sampel sekaligus.

$$\hat{y} = X\theta \quad X \in \mathbb{R}(n \times d), \quad \theta \in \mathbb{R}^d, \quad \hat{y} \in \mathbb{R}^n$$

Contoh Manual

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 3 & 3 \end{bmatrix}, \quad \theta = [2, 1]^T$$

$$\hat{y}_1 = (1)(2) + (2)(1) = 4$$

$$\hat{y}_2 = (2)(2) + (1)(1) = 5$$

$$\hat{y}_3 = (3)(2) + (3)(1) = 9$$

$$\hat{y} = [4, 5, 9]^T$$

Makna

1. Prediksi Sekaligus (Batching)

Satu operasi matriks menghitung prediksi banyak sampel secara bersamaan.

2. Dasar Model Linear

Regresi Linear dan Regresi Logistik menggunakan operasi linear sebagai fondasi perhitungan prediksi.

3. Fondasi Deep Learning

Neural Network menyusun banyak operasi linear dan fungsi aktivasi non-linear secara berlapis.

Segmetation based on Color Channel

Yuita Arum Sari

Multiplication of V and Cb color channel using Otsu thresholding for tomato maturity clustering

Publisher: IEEE

Cite This



Yuita Arum Sari ; Sigit Adinugroho ; Putra Pandu Adikara ; Abidatul Izzah All Authors

9
Cites in
Papers

164
Full
Text Views



Abstract

Document Sections

- I. Introduction
- II. Dataset and Method of Research
- III. Experimental Result and Discussion
- IV. Conclusion

Authors

Figures

References

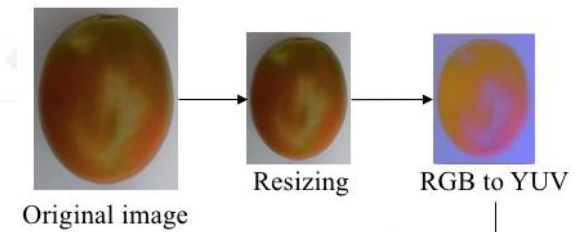
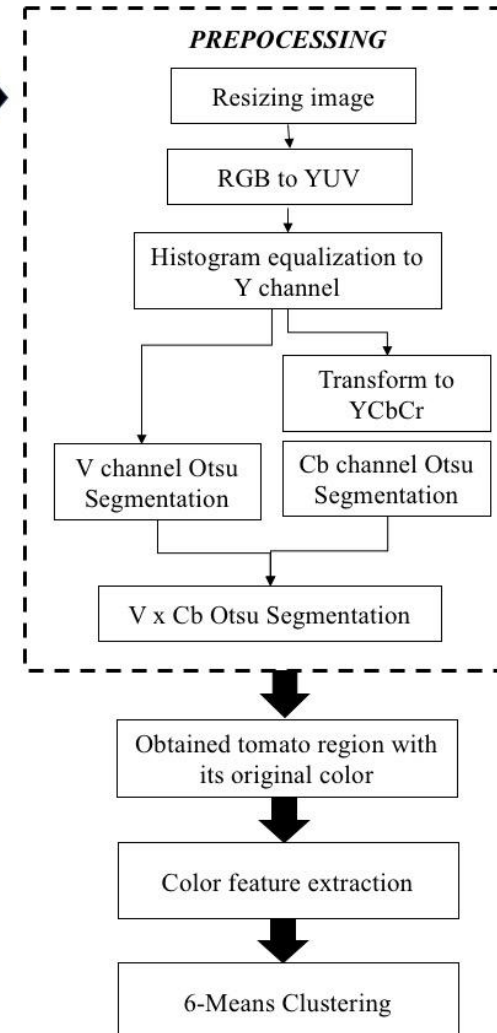
Abstract:

One of the necessary stages of doing digital image processing is laid on the preprocessing phase, in which one of those techniques is segmentation. Segmentation plays an important role in separating a tomato as object and background. When capturing tomato image at outdoor, the segmentation for each device can be different and causes lower performance of maturity clustering problem. This paper proposes new framework in to enhance the evaluation measure of clustering by using combined segmentation of V and Cb color Channel. V color channel is taken from YUV color space, while Cb is obtained from YCbCr color space. Both two color channels are segmented using Otsu thresholding since Otsu is robust to segment each tomato image. Color feature extraction is then applied to the retrieved color of the segmented image. In this paper, RGB and $L^*a^*b^*$ color space model are used to get the feature, but only R, G, a^* , b^* color channel are examined. The 6-Means clustering algorithm is also applied to agglomerate tomato maturity based on six color levels. There are four smartphone cameras to capture the tomato image with unconditional lighting condition at outdoor. The experimental result shows that the smallest value from the average of Mean Square Error (MSE) in all devices reach 3.135. This indicates that the new framework can cluster the tomato maturity with only use G and B color channel.

<https://ieeexplore.ieee.org/abstract/document/8304136>



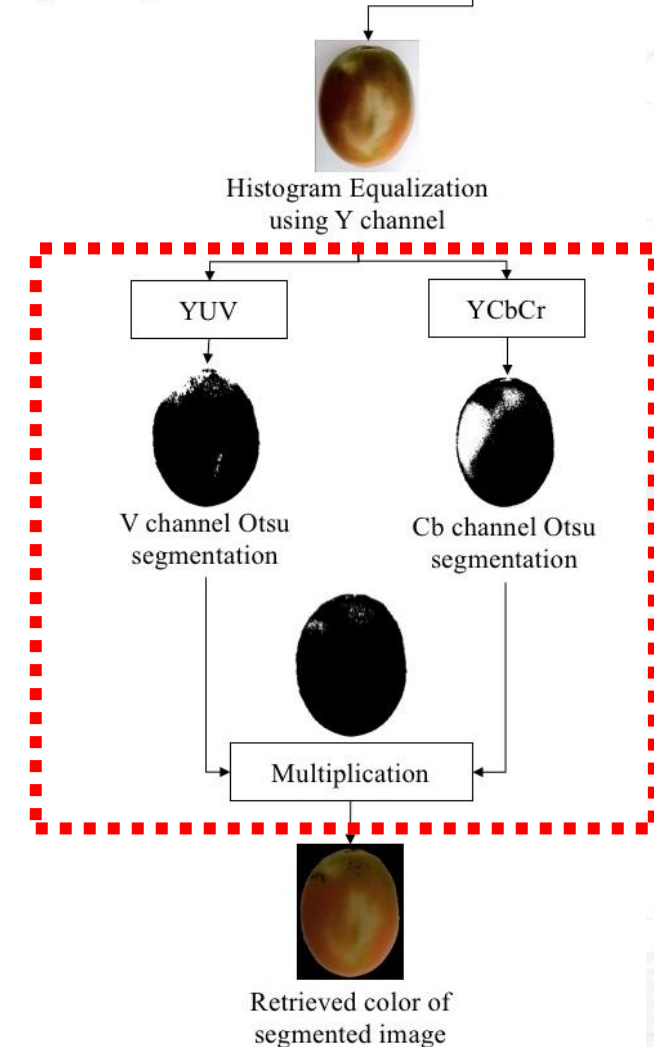
Original image



Original image

Resizing

RGB to YUV



Retrieved color of segmented image

Matriks sebagai Transformasi Ruang

Matriks bukan sekadar tabel angka, tetapi dapat dipandang sebagai fungsi yang mengubah vektor dari satu posisi atau ruang ke posisi/ruang lain.

$$z = Ax$$

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$Ax = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

Interpretasi Geometri

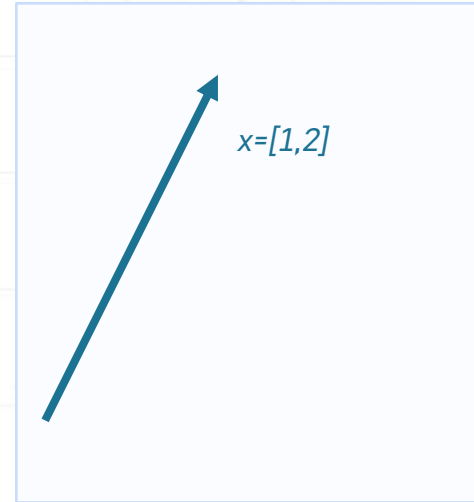
$$(1, 2) \rightarrow (2, 2)$$

Komponen pada **sumbu-X diregangkan 2 kali**, sedangkan komponen pada **sumbu-Y tetap**. Transformasi ini disebut **scaling**.

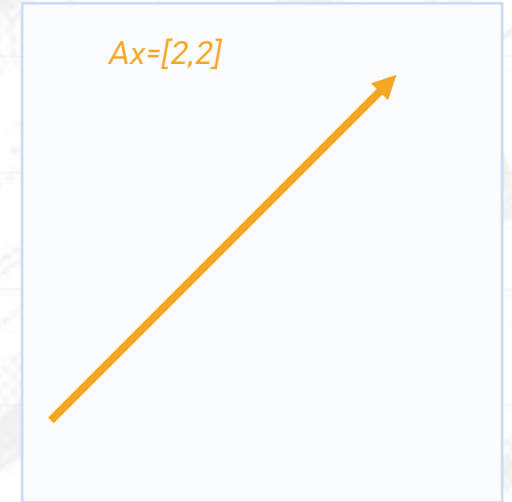
Matriks dapat digunakan untuk:

- **Scaling** → memperbesar atau memperkecil ukuran
- **Rotation** → memutar posisi atau arah vektor
- **Projection** → memetakan data ke ruang atau dimensi lain
- **Transformasi Fitur** → menghasilkan representasi fitur baru

Sebelum



Sesudah (Ax)



Hubungan dengan Deep Learning

$$z = Wx + b$$

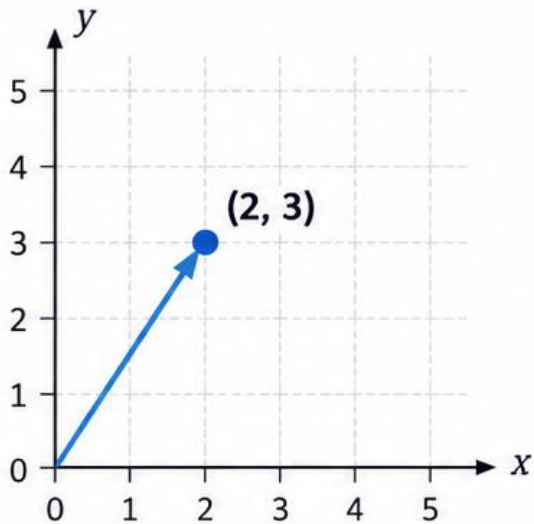
Pada *neural network*, setiap **linear layer** melakukan transformasi matriks terhadap fitur masukan.

Ruang Input $\xrightarrow{W}$ Ruang Fitur Baru $\xrightarrow{\text{Aktivasi}}$ Layer Berikutnya

mengubah representasi data sehingga **pola yang kompleks menjadi lebih mudah dipelajari oleh model**.

Contoh Kasus: Scaling pada Citra Digital

Sebelum



Titik awal: $x = [2, 3]^T$
Koordinat: (2,3)

Transformasi Matriks

$$z = Ax$$

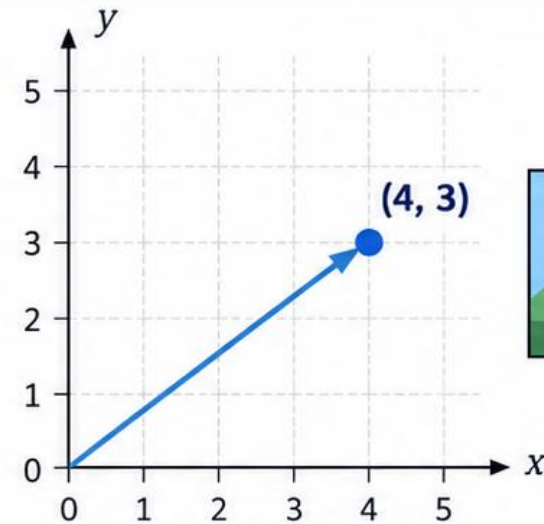
$$A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

- sumbu-X diregangkan 2x
- sumbu-Y tetap



Transformasi scaling
pada sumbu-X

Sesudah



Hasil: $z = [4, 3]^T$
Koordinat baru: (4,3)

Konsep transformasi matriks banyak digunakan pada **resize citra**, **transformasi geometri**, serta **transformasi fitur pada deep learning**.

Sistem Persamaan Linear (SPL): $Ax = b$

Mencari x yang memenuhi sekumpulan persamaan linear.

$$Ax = b$$

A
(Coefficient Matrix)

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$$

×

x
(Unknown Parameter)

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

=

b
(Observed Output)

$$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Perkalian matriks:

$$\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Setara dengan:

$$2x_1 + x_2 = 5 \quad (\text{baris 1})$$

$$x_1 + 3x_2 = 6 \quad (\text{baris 2})$$

Kasus	A (Fitur)	b (Target)	x (Bobot)
Prediksi nilai mahasiswa	Jam belajar, kehadiran, tugas	Nilai akhir	Bobot tiap fitur
Prediksi risiko penyakit	Umur, tekanan darah, BMI	Risiko penyakit	Bobot fitur
Prediksi hasil panen	Curah hujan, pupuk, luas lahan	Hasil panen	Bobot fitur
Prediksi konsumsi BBM	Kecepatan, berat, kapasitas mesin	Konsumsi BBM	Bobot fitur
Prediksi kalori makanan	Berat, protein, lemak, karbohidrat	Kalori	Bobot fitur
Prediksi produktivitas	Jam kerja, pengalaman, jumlah tugas	Produktivitas	Bobot fitur

Yuita Arum Sari Interpretasi Geometris: Perpotongan Garis

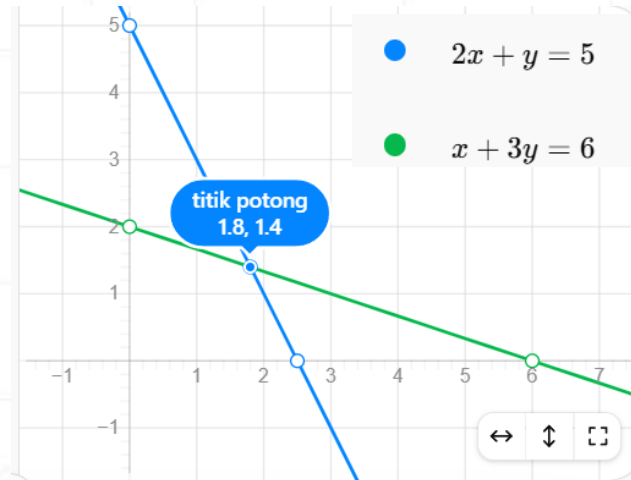
Setiap persamaan linear 2 variabel adalah sebuah garis; solusi sistem adalah titik potongnya.

Dua garis bertemu

Misalnya:

$$2x_1 + x_2 = 5 \text{ dan } x_1 + 3x_2 = 6$$

Masing-masing persamaan menghasilkan **satu garis**.



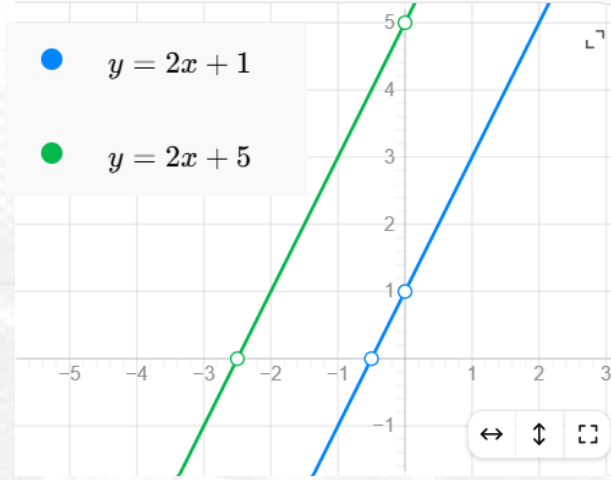
Satu titik yang memenuhi semua persamaan → satu solusi.

Garisnya tidak pernah bertemu

Misalnya: $y = 2x + 1$ dan

$$y = 2x + 5$$

Keduanya mempunyai kemiringan yang sama.



gradien sama → garis sejajar

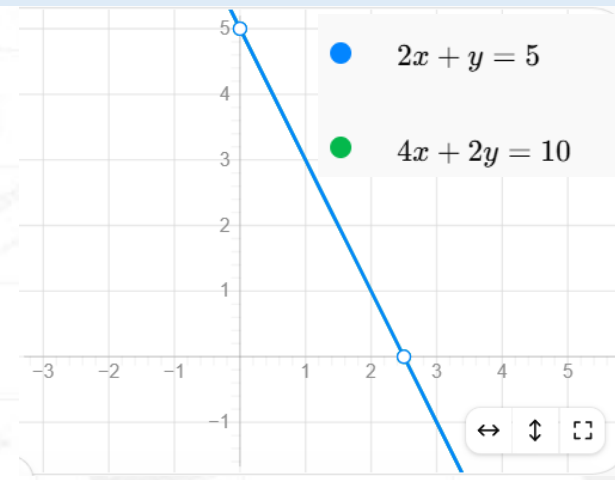
Tetapi, **intercept berbeda** → garisnya tidak sama. Akibatnya, kedua garis **tidak pernah bertemu**. Maka: tidak ada solusi

dua garis ternyata sama persis

Misalnya: $2x + y = 5$ dan

$$4x + 2y = 10$$

Persamaan kedua persamaan pertama yang dikali 2. Menghasilkan **garis yang sama persis**.



Garis sama → titik solusinya tidak terhitung banyak → tak hingga banyak solusi.

SPL: mencari solusi eksak → AI/ML: mencari solusi terbaik dengan error minimum.

Dari SPL ke Training AI/ML

Weight dan bias → model menghasilkan prediksi yang paling sesuai dengan data.

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

Misalnya data training menghasilkan:

$$100w_1 + 3w_2 + b = 500$$

$$150w_1 + 4w_2 + b = 700$$

$$200w_1 + 5w_2 + b = 900$$

Dalam bentuk matriks: $X\theta \approx y$ dengan

$$\theta = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix}$$

Pada data nyata, biasanya **tidak ada solusi eksak**, sehingga ML mencari parameter yang meminimalkan error:

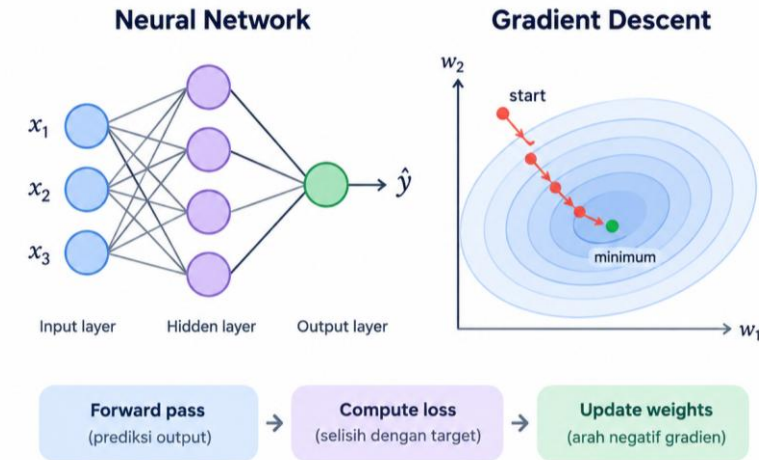
$$\theta^* = \arg \min_{\theta} \|X\theta - y\|^2$$

dan dapat dilakukan dengan **Gradient Descent**:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L$$

Alur:

Data → Model → Loss → Gradient → Update Weight → Model Terlatih



"Weight dan bias harus digeser ke arah mana supaya error semakin kecil?"

- θ = parameter/weight
- L = loss/error
- ∇L = arah perubahan error
- η = **learning rate**, yaitu seberapa besar langkah perubahan

SPL mencari solusi persamaan; ML mencari parameter model yang meminimalkan error terhadap data.

Square System: Jumlah Persamaan = Jumlah Variabel

Jika jumlah persamaan sama dengan jumlah variabel yang dicari, sistem dapat memiliki satu solusi eksak. Jika matriksnya invertible, solusi tersebut dapat dihitung dengan inverse.

Persamaan

2 persamaan, 2 variabel

$$\begin{cases} 2x + y = 5 \\ x + 3y = 6 \end{cases}$$

Matriks

$$\underbrace{\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 5 \\ 6 \end{bmatrix}}_b$$

Inverse

Karena A berbentuk **square** dan invertible:

$$\boxed{x = A^{-1}b}$$

terdapat **satu pasangan** x, y yang memenuhi kedua persamaan sekaligus. Secara geometris, pasangan tersebut adalah **titik potong dua garis**.



Bagaimana jika jumlah data jauh lebih banyak daripada jumlah parameter?

Pada data nyata, biasanya tidak ada solusi yang memenuhi semua persamaan secara eksak

→ **solusi terbaik dengan error minimum (Least Squares).**

Overdetermined System: Persamaan Lebih Banyak dari Unknown

Overdetermined system terjadi ketika jumlah persamaan/data lebih banyak daripada jumlah unknown/parameter. Karena itu, biasanya tidak ada satu parameter yang dapat memenuhi semua persamaan secara tepat.

Misalnya, kita ingin mencari hanya **2 parameter** w_1, w_2 , tetapi memiliki banyak data:

$$\begin{aligned}2w_1 + w_2 &= 5 \\w_1 + 3w_2 &= 6 \\3w_1 + 2w_2 &= 8 \\4w_1 + w_2 &= 10\end{aligned}$$

Ada **4 persamaan** tetapi hanya **2 unknown**.

Dalam bentuk matriks:

$$\underbrace{\begin{bmatrix} 2 & 1 \\ 1 & 3 \\ 3 & 2 \\ 4 & 1 \end{bmatrix}}_{X \ (4 \times 2)} \underbrace{\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}}_{\theta \ (2 \times 1)} \approx \underbrace{\begin{bmatrix} 5 \\ 6 \\ 8 \\ 10 \end{bmatrix}}_{y \ (4 \times 1)}$$

Kenapa memakai $\approx$ dan bukan $=$?

Karena pada data nyata, kemungkinan besar **tidak ada satu nilai** w_1, w_2 yang dapat memenuhi keempat persamaan sekaligus.

Setiap **baris** = satu data training
Setiap **kolom** = satu feature/parameter input

Square System: mencari solusi yang memenuhi semua persamaan
Overdetermined System: mencari parameter yang membuat total error sekecil mungkin

4. Rank and Invertibility

Kapan informasi “hilang”, dan mengapa inverse tidak selalu ada

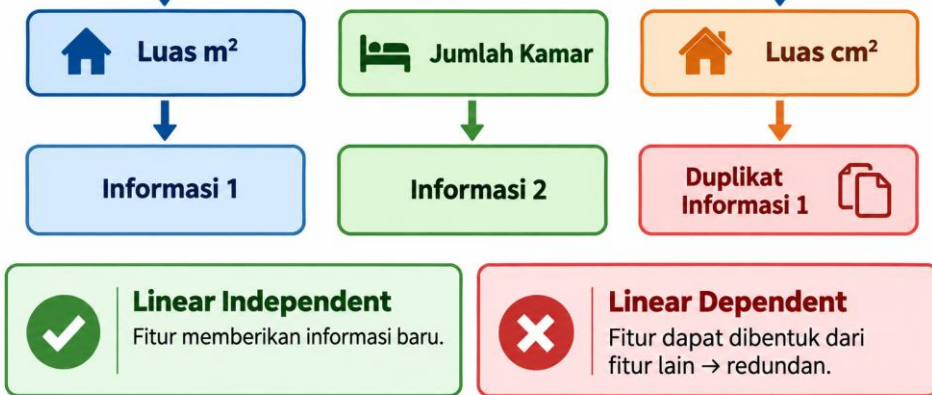
Rank dan Linear Independence:

Apakah Semua Fitur Membawa Informasi Baru?

$\text{rank}(A) = \text{jumlah maksimum kolom/baris yang independen secara linear}$

3 Fitur Input

$\text{Luas}(\text{cm}^2) = 10.000 \times \text{Luas}(\text{m}^2)$



Rank menunjukkan berapa banyak informasi independen yang benar-benar dimiliki data, bukan sekadar berapa banyak kolomnya.

Misalkan terdapat tiga fitur:

$$x_1 = \begin{bmatrix} 50 \\ 70 \\ 100 \end{bmatrix}, x_2 = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}, x_3 = \begin{bmatrix} 500000 \\ 700000 \\ 1000000 \end{bmatrix}$$

dengan:

- x_1 = luas rumah dalam m^2
- x_2 = jumlah kamar
- x_3 = luas rumah dalam cm^2

Matriks fiturnya:

$$X = \begin{bmatrix} 50 & 2 & 500000 \\ 70 & 3 & 700000 \\ 100 & 4 & 1000000 \end{bmatrix}$$

Meskipun X memiliki **3 kolom**, kolom ke-3 hanya duplikasi matematis dari kolom pertama. Maka:

$$\text{rank}(X) = 2$$

Dalam regresi, memasukkan x_1 dan x_3 sekaligus tidak menambah informasi baru; justru dapat membuat penentuan bobot model menjadi tidak stabil.

Rank Deficient: Mengapa Penting ?

Satu konsep aljabar linear yang menjelaskan redundansi fitur, ketidakstabilan parameter, dan reduksi dimensi.

1. Hubungan Matematis

$$\exists a \neq 0: Xa = 0$$



$$\text{rank}(X) < d$$



$$(X^T X) \text{ singular} \\ \Rightarrow (X^T X)^{-1} \text{ tidak ada}$$



$$\beta = (X^T X)^{-1} X^T y \\ \text{tidak unik/tidak terdefinisi}$$

2. Makna dalam Regresi / AI

Jika dua fitur mengukur hal yang sama, model tidak bisa membedakan kontribusinya secara unik.

$$x_3 = 10.000x_1$$

misal: luas(m²) dan luas(cm²)



banyak β memberi prediksi $\hat{y}$ yang sama

- Dependensi linear eksak → parameter tidak unik.
- Dependensi yang hampir eksak → parameter sangat sensitif (tidak stabil).

3. Mengapa Mengarah ke PCA / SVD?

Jika sebagian kolom redundan, maka dimensi efektif data lebih kecil daripada jumlah fitur semula.

$$\text{rank}(X) = r < d$$



Cari arah utama/basis berdimensi r



PCA/SVD membuang arah redundan dan mempertahankan informasi independen

Inti: rank deficient = redundansi linear pada fitur; akibatnya regresi bermasalah, dan karena itu PCA/SVD diperlukan untuk mereduksi dimensi.

Mengapa A^{-1} Tidak Selalu Tersedia?: Jarang dipakai ML

Inverse hanya tersedia pada matrix square yang nonsingular.

1. Bukan Square

Invers standar hanya ada untuk matriks persegi.

$$X \in \mathbb{R}^{100 \times 3}$$

100 sampel, 3 fitur

Bentuknya rectangular, sehingga X^{-1} standar tidak didefinisikan.

$$n \neq d$$

2. Singular/Rank Deficient

Ada fitur yang merupakan kombinasi linear fitur lain.

$$\text{rank}(X) < d$$

contoh: $x_3 = 10.000 x_1$

Akibat pada regresi linear:

$$X^T X \text{ singular}$$

$(X^T X)^{-1}$ tidak ada $\rightarrow$ bobot tidak unik.

3. Ill Conditioned

Secara teori invers ada, tetapi perhitungan sangat sensitif.

$$\det(A) \approx 0$$

Perubahan kecil pada data dapat membuat hasil invers berubah besar.

noise kecil $\rightarrow \beta$ berubah drastis

Masalah ini sering muncul ketika fitur sangat mirip atau redundan.

Karena matriks data X pada ML umumnya rectangular atau dapat mengalami rank deficiency, kita tidak mencari X^{-1} secara langsung. Sebaliknya, kita **mencari parameter yang meminimalkan error melalui Least Squares**, dan jika invers biasa tidak tersedia, menggunakan **Pseudo-Inverse**.

1 X^{-1} tidak tersedia / tidak cocok

Contoh:
 X tidak persegi atau singular ($\text{rank} < n$).

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

($\text{rank} = 2 < 3$)

✗ Inverse tidak ada atau tidak stabil.

2 $X\theta \approx y$

Model linear:

$$X\theta \approx y$$

X : matriks fitur ($n \times d$)
 θ : parameter ($d \times 1$)
 y : target ($n \times 1$)

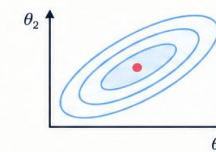
$$\begin{bmatrix} \text{grid} \end{bmatrix} \times \begin{bmatrix} \text{column} \end{bmatrix} \approx \begin{bmatrix} \text{column} \end{bmatrix}$$

$X \quad \theta \quad y$

3 Least Squares

Cari parameter yang meminimalkan error:

$$\min_{\theta} \|X\theta - y\|^2$$



Solusi = titik minimum (error paling kecil)

4 Pseudo-Inverse X^+

Solusi least squares:

$$\theta = X^+ y$$

dengan

$$X^+ = (X^T X)^+ X^T$$

(Moore-Penrose pseudo-inverse)

✓ Memberikan solusi terbaik (minimum norm) meskipun X tidak invertible.

Condition Number: Seberapa Sensitif Model terhadap Gangguan Data?

Makin besar $\kappa(A)$, makin kecil gangguan yang dibutuhkan untuk membuat hasil berubah drastis.

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

indikator sensitivitas numerik

Stabil

Mulai sensitif

Sangat sensitif

$\kappa \approx 1$
Contoh well-conditioned

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$\kappa(I) = 1 \rightarrow$ sangat stabil

gangguan kecil pada data

hasil berubah kecil

Makna Intuitif

$\kappa(A)$ bertindak seperti
“penguat kesalahan”.

Jika κ besar, error kecil pada input
dapat diperbesar menjadi error
besar pada solusi / parameter.

$$\delta A \rightarrow \delta \theta$$

κ sangat besar
Contoh ill-conditioned

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$$

$\kappa(A) \approx 40.000 \rightarrow$ nyaris singular

gangguan kecil / pembulatan

parameter bisa melonjak

Respons Praktis ML

1

Kurangi fitur mirip
atasi multicollinearity

2

Feature scaling
samakan skala fitur

3

Regularisasi L1/L2
tahan bobot agar tidak ekstrem

Kesimpulan: inverse mungkin ada secara teori, tetapi jika $\kappa(A)$ besar, hasilnya tidak dapat dipercaya secara numerik.

5. Least Squares dan Regresi

Mencari solusi terbaik, normal equation, dan model prediktif

Least Squares: Mencari Solusi Terbaik

“Kalau persamaan $Ax = b$ tidak bisa tepat, bagaimana memilih x yang hasilnya paling mendekati b ?”

Misalkan kita ingin: $Ax \approx b$

Karena hasil Ax tidak selalu sama persis dengan b ,
muncul **error (residual)**:

$$r = b - Ax$$

Least Squares memilih x yang membuat **total error sekecil mungkin**:

$$\min_x \|Ax - b\|_2^2$$

Secara sederhana: Data sebenarnya $b \leftrightarrow$ Prediksi Ax Semakin dekat Ax dengan b , semakin kecil errornya.

Data pengamatan:

$$b = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}$$

Model menghasilkan:

$$Ax = \begin{bmatrix} 2.8 \\ 5.2 \\ 6.9 \end{bmatrix}$$

Ada selisih pada setiap data. **Least Squares** mencari parameter x agar jumlah kuadrat semua selisih tersebut minimum.

Hubungan dengan AI/ML:
Linear Regression pada dasarnya mencari parameter model yang meminimalkan error kuadrat antara nilai aktual dan prediksi.

Geometri Least Squares: Projection

Jika Ax tidak tepat sama dengan b , kita mencari Ax yang paling dekat dengan b .

Intuisi sederhana

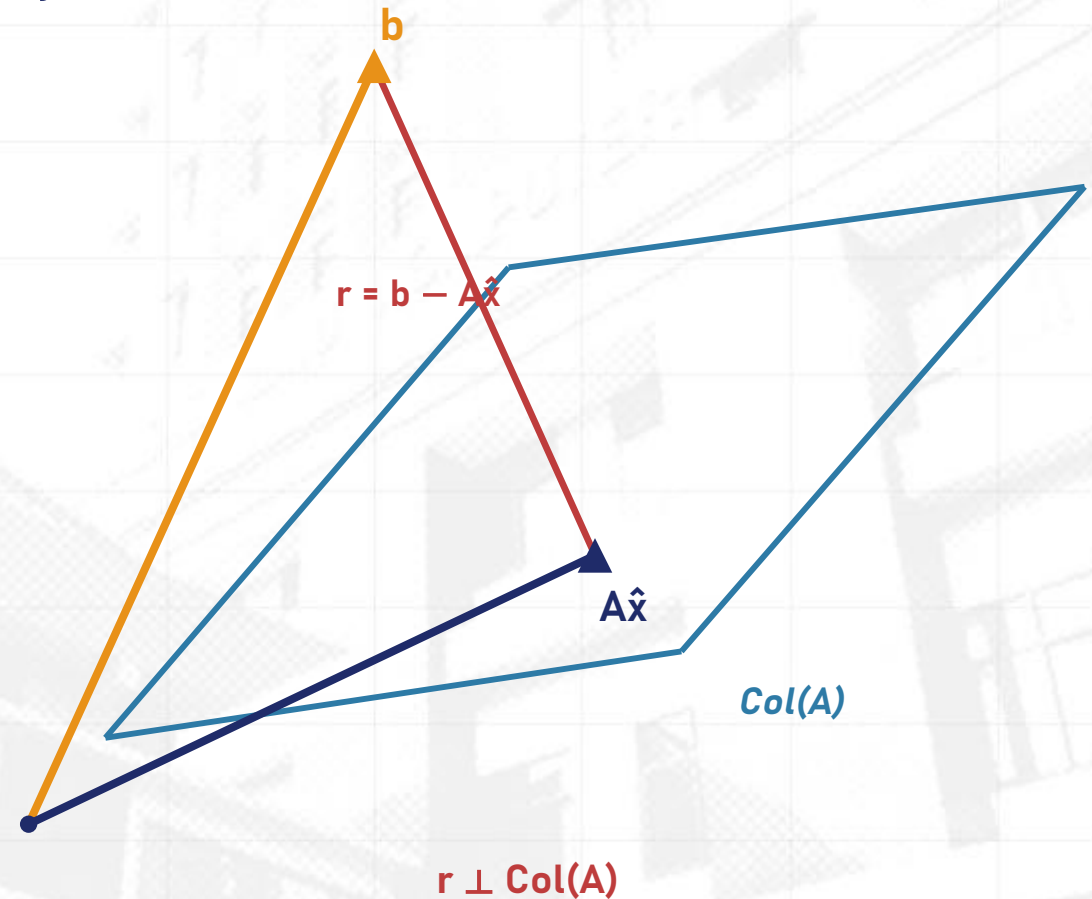
- 1 Kita punya target b dan ingin mendekatinya dengan Ax .
- 2 Tidak selalu ada Ax yang tepat sama dengan b .
- 3 Maka kita pilih $A\hat{x}$ yang jaraknya paling dekat dengan b .

Residual $r = b - A\hat{x}$

Residual adalah jarak / kesalahan

$$A^T(b - A\hat{x}) = 0$$

Proyeksi: mencari titik terdekat



Least Squares = $A\hat{x}$ yang paling dekat dengan b

Manual Least Squares: Kasus Prediksi Kecil

Diberikan tiga data:

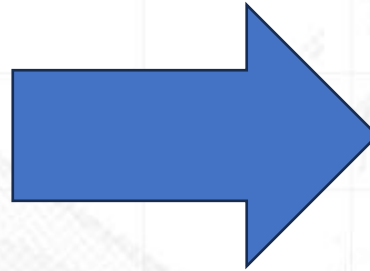
$(1, 2), (2, 3), (3, 5)$

Kita ingin mencari **garis terbaik**:

$$\hat{y} = \theta_0 + \theta_1 x$$

Dalam bentuk matriks:

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}}_X \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\theta} \approx \underbrace{\begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}}_y$$



Dengan Least Squares:

$$\theta = (X^T X)^{-1} X^T y$$

Untuk data ini diperoleh:

$$\theta_0 = \frac{2}{3}, \theta_1 = 1.5$$

sehingga garis prediksinya:

$$\hat{y} = \frac{2}{3} + 1.5x$$

Karena ada **3 data tetapi hanya 2 parameter**, tidak semua titik harus dilewati tepat.

Cari θ yang membuat $X\theta$ sedekat mungkin dengan y

**Data $\rightarrow$ bentuk matriks $X\theta \approx y \rightarrow$ cari parameter
 $\rightarrow$ diperoleh garis yang paling mendekati seluruh data.**

Normal Equation: Rumus untuk Mencari Parameter Terbaik

Tujuannya sederhana: cari parameter θ agar prediksi $X\theta$ sedekat mungkin dengan data y .

1 Kita punya model

$$X\theta \approx y$$

Data dimasukkan ke model untuk menghasilkan prediksi.

2 Cari error terkecil

$$\min \|X\theta - y\|^2$$

Pilih θ yang membuat selisih prediksi dan data sekecil mungkin.

3 Dari kondisi minimum

$$X^T(X\theta - y) = 0$$

Artinya residual tegak lurus terhadap ruang kolom X .

4 Normal Equation

$$X^T X \theta = X^T y$$

Ini adalah persamaan yang digunakan untuk memperoleh θ .

$$\theta = (X^T X)^{-1} X^T y$$

Jika $X^T X$ dapat dibalik, parameter θ dapat dihitung langsung.

INTI: Normal Equation = jalan cepat untuk menemukan parameter Least Squares.

Normal Equation: Dari 3 Data ke Garis Prediksi

Tujuan: mencari garis regresi terbaik $\hat{y} = \theta_0 + \theta_1 x$ tanpa iterasi *gradient descent*.

1 Data

Tiga observasi
(x, y)

x	y
1	2
2	3
3	5

→ satu garis yang paling dekat dengan ketiga titik.

2 Susun matriks

Tambahkan kolom 1

Kolom 1 dipakai untuk θ_0 (intercept).

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$$

$\theta = [\theta_0, \theta_1]^T$
intercept slope

3 Hitung

Normal equation

$$(X^T X) \theta = X^T y$$

$$X^T X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

$$X^T y = \begin{bmatrix} 10 \\ 23 \end{bmatrix}$$

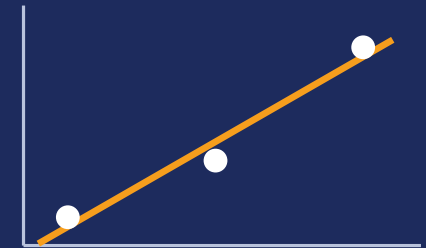
Lalu selesaikan sistem 2x2 tersebut.

4 Dapatkan model

Hasil yang perlu diingat

$$\theta_0 = 1/3 \quad \theta_1 = 3/2$$

$$\hat{y} = 0,333 + 1,5x$$



Garis tidak harus melewati semua titik.

Yang diminimalkan adalah total error kuadrat.

Intuisi

Normal Equation mengubah masalah “cari garis terbaik” menjadi sistem persamaan linear untuk θ_0 dan θ_1 .

Mengapa Kita Membutuhkan Pseudo-Inverse?

Kasus data nyata

3 observasi, 2 parameter

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

X berukuran 3×2 → tidak square.

Jadi X^{-1} tidak ada. Namun kita tetap ingin θ yang membuat $X\theta$ sedekat mungkin dengan y .

Perubahan cara berpikir

$$X\theta = y$$

Solusi eksak

Semua persamaan harus terpenuhi.

$$X\theta \approx y$$

Best fit

Error boleh ada, tetapi totalnya dibuat minimum.

$$\theta = \operatorname{argmin}_{\theta} \|X\theta - y\|^2$$

Gambaran ke AI/ML

- Dataset hampir selalu berbentuk matrix: baris = sampel, kolom = fitur.
- Jumlah sampel n biasanya $\neq$ jumlah fitur $d \rightarrow X$ rectangular.
- Training berarti mencari parameter yang meminimalkan loss, bukan “membalik matrix data”.

DATA → MODEL → LOSS → OPTIMIZE

Benang merah: pseudo-inverse adalah pintu masuk menuju ide training model → mencari parameter terbaik ketika data tidak dapat dipenuhi secara eksak.

Solving Least Squares: Inverse, Pseudo-Inverse, and Gradient Descent

Explicit inverse

Dari normal equation:

$$X^T X \theta = X^T y$$

$$\hat{\theta} = (X^T X)^{-1} X^T y$$

Misalnya:

$$X^T X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

kita hitung inverse-nya, lalu memperoleh θ .

Masalahnya, pada dataset nyata X bisa besar atau kolom-kolomnya sangat mirip.

Pseudo-inverse

$$\theta = X^+ y$$

X^+ dibaca **pseudo-inverse** dari X . Kalau kondisi X baik dan full column rank:

$$X^+ = (X^T X)^{-1} X^T$$

sehingga sebenarnya:

$$X^+ y = (X^T X)^{-1} X^T y$$

Jadi dua kotak pertama **bukan dua tujuan berbeda**.

$$(X^T X)^{-1} X^T y = X^+ y$$

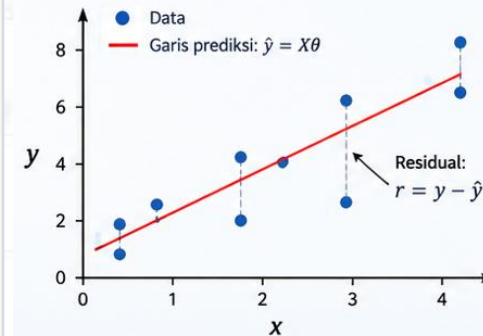
pada kondisi yang sesuai. Kelebihan pseudo-inverse: konsepnya tetap bisa digunakan ketika X tidak square atau memiliki masalah rank.

- Umumnya berbasis SVD.

Masalah:

$$\min_{\theta} L(\theta) = \|X\theta - y\|^2$$

(Least Squares)



(a) Solusi Tertutup (Closed-form) – untuk model linear

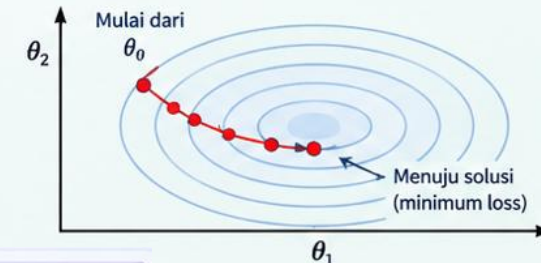
$$\hat{\theta} = (X^T X)^{-1} X^T y = X^+ y$$

(Normal Equation) (Pseudo-Inverse)

(b) Solusi Iteratif – Gradient Descent

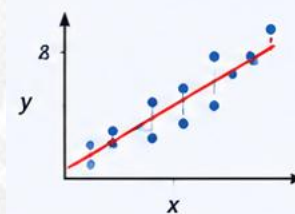
$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

- Tidak perlu menghitung inverse matrix.
- Dapat digunakan untuk model yang lebih kompleks.
- Inti dari banyak algoritma ML modern.



(a) Linear Regression (ada solusi tertutup)

$$\hat{y} = X\theta$$

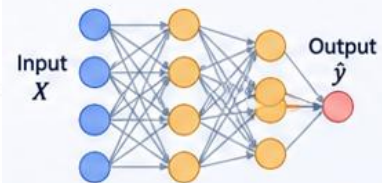


Solver yang bisa digunakan:

- $(X^T X)^{-1} X^T y$
- $X^+ y$
- `lstsq(X, y)`

✓ Solusi tertutup tersedia.

$$\hat{y} = f_{\theta}(X)$$



Tidak ada Solusi tertutup sederhana → parameter dicari secara iteratif

Bagaimana Linear Regression “Belajar”?

Inti machine learning: coba parameter → buat prediksi → ukur error → perbaiki parameter.

Contoh konkret

Data training:

x	y
1	2
2	3
3	5

Model yang dicari:

$$\hat{y} = \theta_0 + \theta_1 x$$

θ_0 dan θ_1 belum diketahui.
Training = mencari nilai keduanya.

θ = parameter yang dipelajari

Training Loop: apa yang dilakukan komputer?

1 COBA MODEL

$\theta_0=0,5; \theta_1=1,2$
→ $\hat{y} = 0,5 + 1,2x$

2 PREDIKSI

untuk $x=2$
 $\hat{y} = 2,9$

4 PERBAIKI θ

ubah θ agar
loss lebih kecil

3 HITUNG ERROR

target $y=3$
error = $3 - 2,9 = 0,1$

LOSS
$$L(\theta) = \sum (y_i - \hat{y}_i)^2$$

Semakin kecil loss → parameter semakin baik.

Kapan training berhenti?

Cari parameter terbaik

$$\theta = \operatorname{argmin} L(\theta)$$

= θ yang membuat loss minimum

Hasil contoh

$$\hat{y} = 0,333 + 1,5x$$

Koneksi ke AI/ML

Linear regression:
 θ dicari dengan least squares / solver.

Neural network:
weights dicari secara iteratif.

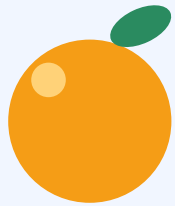
Gradient Descent + Backpropagation

MODEL membuat prediksi → LOSS mengukur kesalahan → TRAINING mengubah parameter agar loss mengecil.

Satu Fitur → Garis, Banyak Fitur → Hyperplane

Contoh citra: memprediksi tingkat kematangan buah dari fitur visual.

1 FITUR → SIMPLE REGRESSION



FITUR x

Rata-rata
warna merah

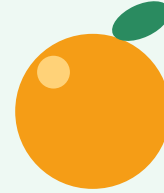
Kematangan
 $\hat{y}$

$$\hat{y} = \theta_0 + \theta_1 x$$

1 fitur → titik data dapat digambar sebagai GARIS



BANYAK FITUR → MULTIPLE REGRESSION



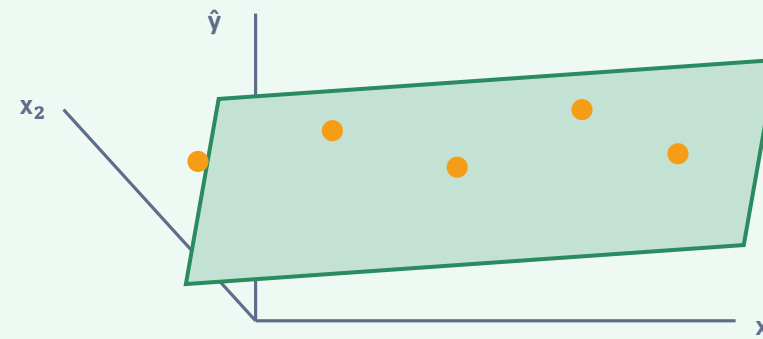
x_1
Rata-rata
Red

x_2
Rata-rata
Green

x_3
Tekstur
GLCM

$\hat{y}$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$



Bentuk umum

$$\hat{y} = X\theta$$

dot product

3 fitur → bidang; d fitur → HYPERPLANE

Yang berubah hanya jumlah fitur. Mekanisme tetap: fitur $\times$ bobot → prediksi | Neural Network: $z = XW + b$

Color Correction

Color correction using improved linear regression algorithm

Publisher: IEEE

Cite This

PDF

Yuita Arum Sari ; R. V. Hari Ginardi ; Nanik Suciati All Authors

9

Cites in
Papers

614

Full
Text Views



Abstract

Document
Sections

I. Introduction

II. Method of
Research

III. Results
and
Discussion

IV. Conclusion

Abstract:

Color correction is one of essential stages in image processing, which plays an important role during image acquisition or pre-processing to produce a better color quality, before being used in further process. This paper proposes a new method for color correction using an improved linear regression algorithm based on a stepwise model. This proposed method is designed for assessing a series of discrete color levels, for instance in a leaf color chart. Color chart as a reference image is used for controlling color levels of a captured image or calibrating the image sensor. The experiment is conducted in L*a*b* color space, therefore a transformation from RGB into L*a*b* is needed at the first phase. The best matched color level between reference and captured image will be selected by k-Means clustering

Diff = 29.244
Entropy = 6.981

Diff = 11.317
Entropy = 6.778

Diff = 30.338
Entropy = 6.513

Original Image



Color Correction Image



Diff = 2.291
Entropy = 6.937

Diff = 0.037
Entropy = 6.819

Diff = 0.070
Entropy = 6.394

<https://ieeexplore.ieee.org/abstract/document/7379874>

Dari Residual ke Loss dan Metric: Apa Bedanya?

Mulai dari satu prediksi yang salah, lalu lihat bagaimana error dipakai untuk training dan evaluasi.

CONTOH 1 DATA

Target sebenarnya

$$y = 5$$

Prediksi model

$$\hat{y} = 4$$

Residual = selisih

$$r = y - \hat{y} = 1$$

Untuk loss: kuadratkan error

$$r^2 = (5 - 4)^2 = 1$$

Residual adalah error SATU sample. Untuk banyak sample, error-error ini dikumpulkan menjadi LOSS.

1 TRAINING — “Bagaimana model belajar?”

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2$$

Training mengubah parameter θ agar total error semakin kecil.

Prediksi

Loss

θ

2 EVALUATION — “Seberapa baik modelnya?”

MAE

rata-rata
|error|

RMSE

error besar
lebih terasa

R^2

variasi yang
dijelaskan

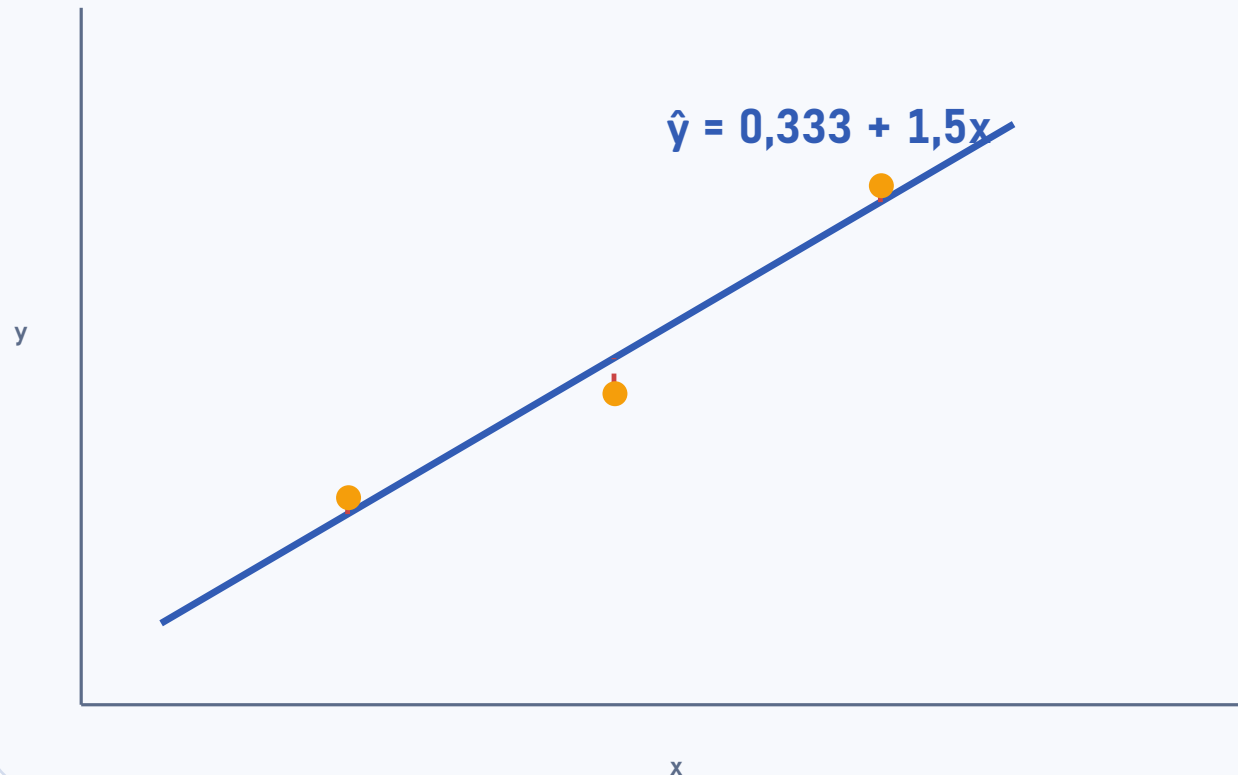
Nilai metric sebaiknya dihitung pada validation/test data — bukan hanya data training.

RESIDUAL = salah per data → **LOSS = angka yang diminimalkan saat training** → **METRIC = nilai untuk menilai model**

Mengapa Garis Ini Dipilih? Lihat Residualnya

Contoh yang sama: (1,2), (2,3), (3,5) dengan $\hat{y} = 0,333 + 1,5x$.

Data, garis prediksi, dan residual vertikal



Hitung error per titik

$$x=1 \quad \hat{y}=1,833 \quad r=+0,167$$

$$x=2 \quad \hat{y}=3,333 \quad r=-0,333$$

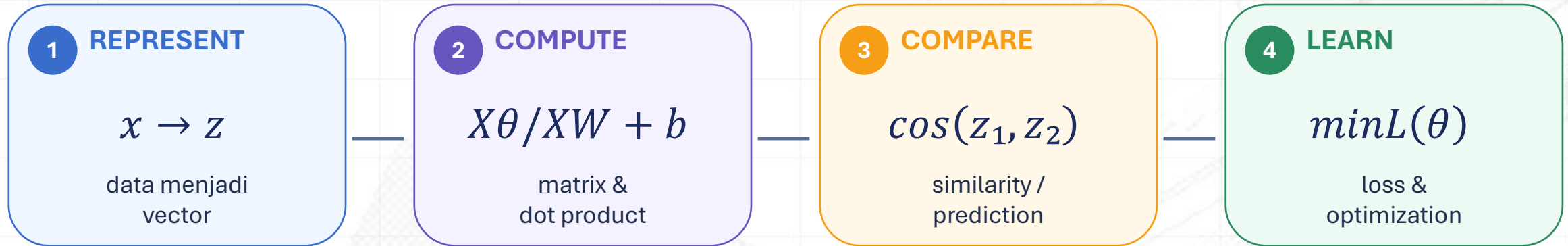
$$x=3 \quad \hat{y}=4,833 \quad r=+0,167$$

$$\text{SSE} \approx 0,167$$

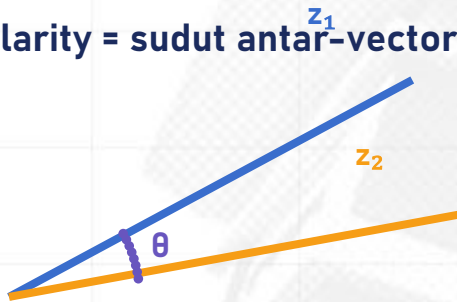
Least squares memilih garis dengan total panjang residual² paling kecil.

Matematika yang Sama di Balik AI

Model boleh berubah dari regresi hingga deep learning, tetapi bahasa dasarnya tetap vector, matrix, similarity, loss, dan optimization.



Cosine similarity = sudut antar-vector



arah mirip → cosine mendekati 1 |
arah berbeda → similarity turun

Satu matematika, banyak aplikasi AI

NLP / RAG
teks → embedding
→ cari vector terdekat

Image Retrieval
citra → embedding
→ cari citra serupa

Multimodal
teks ↔ gambar
→ ruang vector bersama

Yang dibandingkan bukan data mentah, tetapi representasi vector-nya.

VECTOR → MATRIX → DOT PRODUCT / SIMILARITY → LOSS → OPTIMIZATION

Terima Kasih

Yuita Arum Sari, S.Kom., M.Kom., Ph.D.
yuita@ub.ac.id

Versi Dokumen:

Tanggal: 15; Bulan: September; Tahun: 2026

© Fakultas Ilmu Komputer, Universitas Brawijaya



filkom.ub.ac.id